Analyseurs syntaxique et sémantique pour un sous-langage de TypeScript

Maxence CHEVÉ

1 Utilisation

Afin de pouvoir tester un programme, il faut d'abord executer la commande

```
make
```

Cela va générer l'exécutable `tp_parser` dans le dossier `bin`. Il suffit alors d'exécuter le programme en donnant le fichier contenant le script en entrée :

```
./bin/tp_parser script.ts
```

2 Structure Générale du Projet

Le projet est divisé en plusieurs modules, chacun jouant un rôle précis dans le processus de compilation :

- `lexer.mll` : Analyse lexicale, responsable de transformer le code source en une suite de *tokens*.
- `parser.mly` : Analyse syntaxique, construisant l'AST à partir des *tokens*.
- `ast.ml` : Définition des structures de données pour représenter l'AST. Et mise en place de la fonction afin de transformer un AST en un code JavaScript.
- `main.ml` : Point d'entrée du programme, orchestrant les différentes étapes de compilation. Avec les définitions des fonction `check_scope` et `check_type`.

3 Analyse Lexicale : `lexer.mll`

Le fichier `lexer.mll` réalise une analyse lexicale du script passé en entrée et génère des token pour construire l'AST avec `lexer.mll`.

3.1 Problèmes rencontrés

Difficultés à gérer les cas des commentaires bloc, des `string` entre guillemets simples et `string` entre guillemets doubles avec de simples regex.

3.2 Solution Trouvée

Utiliser des fonctions auxiliaires qui démarrent lorsque le premier caractère définissant un `string` ou un commentaire bloc. Ces fonctions s'arrêtent uniquement lorsque le caractère terminal est atteint (en ne prenant pas en compte les caractères précédés d'un \ dans le cas des `string`).
Dans le cas des commentaires block, le contenu est ignoré.

4 Analyse Syntaxique : `parser.mly`

Le fichier `parser.mly` construit l'AST en fonction de l'analyse lexicale réalisée par `lexer.mll`.

4.1 Problèmes rencontrés

L'organisation du fichier parser.mly m'a confronté à plusieurs problèmes :

- Le cas des listes d'arguments (liste d'éléments d'un type objet, liste de bindings, ...).
- Le type Union qui génère une grammaire cyclique.
- Les conflits décaler/réduire et réduire/réduire.

4.2 Solutions trouvées

J'ai donc trouvé les solutions suivantes :

- Créer un type auxiliaire représentant les listes (`binding_list`, `objectType_list`, ...).
- Créer le choix entre le type union et les autres types. Le type union est alors composé de ces autres types mais un type union ne peut pas être composé d'un autre type union.
- L'ajout de priorités et de règles concernant les token avec `%left`, `%right`, `%nonassoc` et `%prec`.

5 Arbre de Syntaxe Abstraite : `ast.ml`

Le fichier `ast.ml` définit les types pour représenter le programme de manière structurée. Aucun réel problème n'a été rencontré. La conversion en JavaScript est réalisée en créant un `string` pour chaque type défini dans `ast.ml`.

6 Analyse du scope : `check_scope`

Le fichier `main.ml` implémente `check_scope` qui s'occupe de la gestion des portées dans le langage, en vérifiant la déclaration et l'utilisation des variables et des fonctions.

6.1 Problèmes rencontrés

Concernant cette fonction, j'ai rencontré un certain nombre de problèmes :

- Gestion des portées imbriquées dans les blocs et les fonctions.
- Détection des variables utilisées avant leur déclaration.
- Manipulation des constantes pour empêcher leur réaffectation.
- Complexité dans la recherche des variables dans des portées imbriquées, en remontant la pile des portées.

6.2 Solution trouvée

Pour résoudre ces problèmes, plusieurs approches ont été mises en œuvre :

- Utilisation d'une pile de tables de hachage (`Hashtbl.t`) pour représenter les portées. Chaque nouvelle portée est ajoutée sur la pile lorsqu'un nouveau bloc ou une fonction est rencontré, et retirée à la fin du bloc.
- Une fonction récursive pour rechercher une variable en remontant la pile des portées.
- Une gestion explicite des déclarations et des utilisations des variables. Les variables sont marquées comme `is_declared` ou `is_const`, et des exceptions sont levées si une variable est utilisée avant d'être déclarée ou si une tentative de modification d'une constante est détectée.
- Déclenchement explicite des erreurs via des exceptions personnalisées (`ScopeError`) pour fournir des messages clairs en cas de violation des règles de portée.

La fonction `check_scope` a été testée et fonctionne correctement pour gérer les portées imbriquées et détecter la plupart des erreurs de portée. Cependant, des limitations subsistent, notamment dans le traitement des déclarations avec `var`.

7 Analyse de type : `check_type`

La fonction `check_type` est responsable de la vérification des types dans le langage.

7.1 Problèmes rencontrés

J'ai rencontré certains problèmes lors de la création de `check_type` :

- Gestion des environnements pour suivre les types des variables et des fonctions.
- Vérification de la compatibilité des types dans des contextes complexes, comme les structures imbriquées (objets, tableaux, unions).
- Implémentation d'une logique de sous-typage pour gérer les constantes, les unions et les types primitifs.
- Difficulté à traiter les erreurs liées au type de retour dans les fonctions.

7.2 Solution trouvée

J'ai donc essayé d'implémenter plusieurs solution :

- **Environnement typé** : Un environnement basé sur une table de hachage (module `Env`) permet de mapper les identifiants à leurs types respectifs.
- **Sous-typage** : Une fonction récursive `is_subtype` vérifie si un type est compatible avec un autre, en prenant en compte les particularités des types unions et des types primitifs.
- **Vérification récursive** : Les expressions, instructions, et déclarations sont vérifiées de manière récursive, chaque élément étant validé en fonction de son contexte.
- **Gestion des erreurs** : Les erreurs de typage sont capturées par des exceptions `TypeError`.

Cependant, cette fonction comporte encore beaucoup de problèmes que je n'ai pas eu le temps de résoudre. Je recommande donc de ne pas l'utiliser pour pouvoir tester les autres fonctions.

8 Point d'Entrée : `main.ml`

Le fichier `main.ml` initialise le lexer et le parser, et gère les erreurs éventuelles. Voici les principales étapes :

- Chargement du fichier source.
- Passage par le lexer pour produire les *tokens*.
- Passage par le parser pour construire l'AST.
- Gestion des erreurs lexicales et syntaxiques.

9 Problèmes non résolus

9.1 Conflit dans le fichier `parser.mly`

Le fichier `parser.mly` contient un conflit décaler/réduire sur la règle suivante :

```
type__ -> type__ LBracket RBracket
```

Ce conflit n'a pas été résolu, ce qui entraîne une limitation importante :
Si un type complexe contient un type tableau (par exemple, `t[]`), cela provoquera une erreur lors de l'analyse syntaxique.
Cette limitation peut restreindre l'utilisation correcte de certains types complexes dans le programme.

9.2 Problème avec la fonction `check_scope`

La fonction `check_scope`, qui est censée gérer la portée des déclarations de variables, ne fonctionne pas correctement dans le cas des déclarations avec `var`. Cela peut entraîner des erreurs ou des comportements inattendus dans le programme lors de la vérification des portées.

9.3 Problèmes avec la fonction `check_type`

La fonction `check_type` présente plusieurs limitations majeures :

- Elle ne fonctionne quasiment pas et détecte fréquemment des erreurs qui n'en sont pas.
- Ces problèmes sont dus à une mauvaise utilisation et compréhension des types dans le programme.

Pour pallier ces problèmes temporaires, la fonction a été désactivée afin de permettre le test des autres parties du programme sans interférence. Pour réactiver cette fonction, il suffit de la décommenter dans le fichier `main.ml` à la ligne 342 :

```
(*check_type ast;*)
```

9.4 Affichage des erreurs

Des efforts ont été faits pour afficher des erreurs précises, mais il reste un problème notable : L'endroit exact où l'erreur est détectée n'est pas bien indiqué dans le cas d'une erreur de lexer et pas affichée dans les autres types d'erreurs, ce qui complique le débogage.

9.5 Affichage des commentaires dans la convertion des AST en JavaScript

N'ayant pas sauvegardé le contenu des commentaires, je n'ai pas pu les ajouter à l'AST. Ce qui implique que je n'ai pas pu les ajouter au code JavaScript.